

Análisis de la precisión de las mediciones del CODIGO

Introducción

En este documento explico, a partir de la revisión del código, por qué considero que las mediciones realizadas por el ESP32 son confiables y más precisas que intentar medir los tiempos manualmente. La principal ventaja es que el microcontrolador registra los eventos directamente desde los botones y el LED, utilizando un temporizador interno de alta resolución. Por eso, no depende de que una persona mire un reloj o intente calcular el tiempo por su cuenta.

1. Medición de los tiempos de reacción (Juego 1)

Al revisar el Juego 1, observé que el programa trabaja con dos tiempos de reacción diferentes. Los dos se obtienen tomando una marca de tiempo al ocurrir un evento y otra marca cuando ocurre el evento siguiente. Para esto se utiliza `esp_timer_get_time()`.

Esta función pertenece al sistema del ESP32 y trabaja en microsegundos desde que el chip inicia. No es una variable que el programa vaya aumentando manualmente, sino un temporizador que continúa funcionando independientemente de la lógica del juego. Por eso, al restar dos lecturas se obtiene el tiempo que realmente pasó entre ambos eventos.

Por ejemplo, el código hace algo equivalente a:

```
tiempo_led_encendido = esp_timer_get_time();
tiempo_suelta_boton1 = esp_timer_get_time();
tiempo_us = tiempo_suelta_boton1 - tiempo_led_encendido;
```

La primera reacción corresponde al intervalo desde que se enciende el LED hasta que el sistema detecta que el jugador libera el Botón 1. La segunda corresponde al tiempo desde que se libera el Botón 1 hasta que se detecta la pulsación del Botón 2.

La lectura de los botones se revisa dentro del ciclo principal, que trabaja cada 5 milisegundos. Por lo tanto, existe un pequeño margen de detección relacionado con ese ciclo, pero sigue estando en el orden de los milisegundos. Esto es mucho más consistente que intentar medir el mismo evento con una persona utilizando un cronómetro.

2. Por qué el promedio de 10 jugadas es correcto

También revisé la forma en que el programa calcula el promedio de la Reacción 2. El sistema no calcula ese valor de manera aproximada. Primero guarda las mediciones obtenidas en el arreglo `tiempos_reaccion2` y después espera a tener diez datos válidos antes de calcular el promedio.

El código utiliza un índice para saber en qué posición guardar cada resultado. Cuando llega al final del arreglo, el índice vuelve a cero. Esto funciona como un buffer circular y evita que se escriban datos fuera del espacio reservado.

Además, la variable `cantidad_reacciones2` permite saber cuántas mediciones válidas se han acumulado. El promedio solamente se calcula y se publica cuando se alcanzan las diez mediciones requeridas. De esta forma, el resultado enviado por MQTT corresponde a un conjunto completo de diez jugadas y no a un promedio realizado con menos datos.

3. Por qué las pulsaciones no se cuentan varias veces

Una parte importante que encontré al analizar el programa es la detección de una pulsación. El problema de leer un botón continuamente es que, si una persona lo mantiene presionado durante varios ciclos, el programa podría interpretarlo como varias pulsaciones.

Para evitar esto, el código compara el estado actual del botón con el estado que tenía en la lectura anterior. La condición utilizada es similar a:

```
bool nueva_pulsacion_boton1 = (boton1_anterior == SUELTO) && (boton1 == PRESIONADO);
```

Con esta lógica, solamente se registra una nueva pulsación cuando el estado cambia de SUELTO a PRESIONADO. Si el botón continúa presionado durante las siguientes vueltas del programa, ya no se vuelve a contar porque el estado anterior también será PRESIONADO.

Al finalizar cada ciclo, el programa actualiza `boton1_anterior` y `boton2_anterior` con los valores actuales. Al analizarlo, entiendo que esta técnica permite detectar el cambio real de la señal digital y no simplemente comprobar si el botón permanece presionado. A esto se le conoce como detección de flanco.

Esta misma idea ayuda en el Juego 1 con la variable `boton1_suelto`, ya que sirve para impedir que una misma reacción sea registrada más de una vez durante la misma jugada.

4. Por qué el conteo de pasos del Juego 2 es válido

En el Juego 2 el objetivo es alternar entre el Botón 1 y el Botón 2 durante un tiempo determinado. Para controlar esto, el programa utiliza la variable `ultimo_boton`, que guarda cuál fue el último botón considerado válido.

La lógica revisa primero si el botón actual es igual al último botón registrado. Si son iguales, no se suma ningún paso. En cambio, cuando se produce una alternancia $B1 \rightarrow B2$ o $B2 \rightarrow B1$, entonces se incrementa `pasos_juego2`.

Esto me permite concluir que una persona no puede aumentar el conteo simplemente presionando varias veces el mismo botón consecutivamente. Para que el contador

aumente debe existir un cambio real entre los dos botones. Así, el número de pasos representa las alternancias válidas realizadas durante el juego.

5. La máquina de estados evita conteos incorrectos

Otro punto que considero importante es que el programa está organizado mediante una máquina de estados. Entre los estados utilizados están ESPERANDO_INICIO, ESPERANDO_LED, ESPERANDO_ACCION y JUEGO_TERMINADO. Esto hace que cada parte del código se ejecute solamente cuando corresponde.

Por ejemplo, la función encargada de reaccionar al Botón 2 no debe registrar una reacción antes de que se haya producido la liberación del Botón 1. Por eso se comprueba primero que boton1_suelto sea verdadero.

También existe un control para una falsa salida. Si el jugador suelta el Botón 1 antes de que aparezca la señal del LED, el programa no toma esa acción como una reacción válida, sino que la identifica como una salida anticipada.

Al finalizar una partida también se espera a que los dos botones estén sueltos antes de comenzar otra. Esto evita que una pulsación que todavía estaba activa de la ronda anterior se interprete como parte de la siguiente. Desde mi análisis, esta organización ayuda bastante a evitar mediciones fuera de contexto o eventos repetidos.

6. Variables principales que intervienen en las mediciones

Después de revisar el funcionamiento general, estas son las variables que considero más importantes para que los resultados sean correctos:

- tiempo_led_encendido y tiempo_suelta_boton1: guardan las marcas de tiempo de los eventos principales usando esp_timer_get_time(). Al restarlas se obtiene el tiempo transcurrido entre ambos eventos.
- boton1_suelto: indica que la primera reacción ya fue registrada. Esto evita que esa misma reacción vuelva a procesarse durante la misma jugada.
- tiempos_reaccion2[10] e indice_reaccion2: almacenan las mediciones de Reacción 2 y controlan la posición donde se guarda cada nuevo resultado. El índice vuelve al inicio cuando llega al final del arreglo.
- cantidad_reacciones2: indica cuántas mediciones válidas se han acumulado. El promedio se realiza cuando se completan las diez jugadas necesarias.
- boton1_anterior y boton2_anterior: guardan los estados anteriores de los botones. Al compararlos con la lectura actual se puede identificar una nueva pulsación en lugar de contar repetidamente un botón mantenido.
- ultimo_boton: se utiliza en el Juego 2 para recordar el último botón válido. Esto obliga a que exista una alternancia entre los botones para que el contador aumente.

7. Conclusión

Después de analizar el código, considero que las mediciones realizadas por el ESP32 son confiables porque los valores no se generan de manera aleatoria ni se calculan manualmente. Los datos vienen directamente de los eventos físicos que el microcontrolador detecta.

La precisión de los tiempos se consigue principalmente mediante `esp_timer_get_time()`, que permite trabajar con marcas de tiempo en microsegundos. Por otro lado, la detección de cambios de estado en los GPIO evita que una misma pulsación sea contada varias veces. Finalmente, la máquina de estados controla en qué momento puede producirse cada acción y evita mezclar eventos de diferentes partes del juego.

Por estas razones, al comparar este método con una medición hecha por una persona, considero que el ESP32 ofrece una medición mucho más objetiva y repetible. Una persona tiene que reaccionar visualmente y utilizar un cronómetro, mientras que el microcontrolador registra directamente las señales eléctricas de los botones y del LED. El pequeño margen producido por el ciclo de lectura de 5 ms es conocido y forma parte del funcionamiento del sistema, por lo que el resultado sigue siendo mucho más consistente para este tipo de prueba.